Proprietary

RAZVOJ MOBILNIH APLIKACIJA I SERVISA

Predavanje br. 2

Uvod u mobilno računarstvo · Uvod u Kotlin (1. dio)

Akademska 2026/27 · Fakultet elektrotehnike, Univerzitet u Tuzli

Proprietary

PLAN PREDAVANJA

Danas: kontekst, pa kod

DIO 1 · ~90 MIN

Uvod u mobilno računarstvo

- Historija i evolucija uređaja
- 3-slojna arhitektura
- Mobilni ekosistem i razvojna okruženja
- Pristupi razvoju i faktori
- Nove dimenzije mobilnog računarstva

DIO 2 · ~90 MIN

Uvod u Kotlin (1. dio)

- Varijable, tipovi, stringovi
- Uslovi, petlje, kolekcije
- Funkcije
- Null sigurnost i lambde
- Vježba: čitanje i review koda

Proprietary

Zašto razvijamo mobilne aplikacije?

60%

web saobraćaja u svijetu dolazi sa mobilnih uređaja

60,04 % mobilni vs 39,96 % desktop · septembar 2026 · izvor: StatCounter Global Stats

Pitanje: u septembru 2024. bilo je 63,38 %. Zašto bi udio mogao pasti?

Proprietary

OSNOVNI POJMOVI

Šta je mobilno računarstvo?

Korištenje računarskih resursa i usluga **u pokretu** — bez fiksne lokacije i fiksne mrežne veze. Razvoj mobilnih aplikacija je kreiranje softvera za pametne telefone, tablete i druge mobilne uređaje.

Hardver

Uređaj: ekran na dodir, senzori, kamera, GPS, baterija

Komunikacija

Mreža: 4G/5G, Wi-Fi, Bluetooth, NFC

Softver

OS, aplikacije i servisi u oblaku

Proprietary

TRŽIŠTE · SEPTEMBAR 2026

Android dominira, posebno kod nas

SVIJET

69 %

Android · iOS 31 %

BOSNA I HERCEGOVINA

76 %

Android · iOS 24 %

Telefoni se koriste za sve: komunikaciju, zabavu, kupovinu, bankarstvo, rad, učenje i dr.

Izvor: StatCounter Global Stats, Mobile OS Market Share, sept. 2026 (Svijet: 69,17 / 30,81 · BiH: 75,67 / 24,32)

Proprietary

EVOLUCIJA UREĐAJA

Pet era mobilnih telefona

1983 →

Brick

Veliki, teški, samo pozivi. Motorola DynaTAC.

1990-e

Candy bar

Manji, SMS, prve igrice. Nokia.

2000-e

Feature

Kamera, MP3, WAP, preklopni dizajn.

~2002 →

Smartphone

Tastatura, e-mail, PDA. BlackBerry, Palm Treo.

2007 →

Touch

Multi-touch, app store, bez dugmadi. iPhone, Android.

Danas: sklopivi uređaji (foldables), tableti, satovi i drugi wearables — jedna aplikacija, mnogo formata ekrana.

Proprietary

KVIZ

Prvi komercijalni mobilni telefon: Motorola DynaTAC 8000X

Godina?

???

Težina?

???

Veličina?

???

Cijena?

???



Proprietary

ODGOVOR

Prvi komercijalni mobilni telefon: Motorola DynaTAC 8000X

Godina

1983

Težina

790 g

Veličina

44 × 330 × 89 mm

Cijena

$3.995

+ mjesečna pretplata + plaćanje po minuti



KRATKI PREGLED · 1983–2002

Od telefona do „računara u džepu“

1983

Motorola predstavlja prvi komercijalni mobilni telefon.

1994

IBM Simon — prvi „pametni“ telefon: ekran na dodir, faks i e-mail.

1997

Nokia 6110 — prva ugrađena igrica, Snake.

2002

Palm Treo — telefon, PDA i e-mail u jednom uređaju.

Šta se promijenilo? Bolje baterije, bolja pokrivenost i povoljniji paketi. Proizvođači nisu htjeli sami pisati sve aplikacije — ali nisu htjeli ni otvoriti platforme.









Proprietary

PRIJE APP STORE-A

Mobilni web i SMS ekonomija

WAP

- Wireless Application Protocol — pojednostavljeni HTTP za nepouzdanu mobilnu mrežu
- WML umjesto HTML-a, koncept „kartica“
- Popularne WAP stranice: CNN, ESPN

Kupovina putem SMS-a

- Poruka na broj s posebnom tarifom → melodija zvona, pozadina, igrica
- Operater kontroliše šta se nudi i uzima dio prihoda
- Svaki proizvođač — svoj OS, teško za aplikacije trećih strana

Proprietary

KRATKI PREGLED · 2007 → DANAS

Moderna era: platforme, prodavnice, alati



2007
iPhone multi-touch interfejs

2008
App Store, Android Market i prvi Android telefon



2010
iPad popularizuje tablete

2014
Material Design — Googleov jezik dizajna

2017
Google uvodi službenu podršku za Kotlin

2019
Android postaje „Kotlin-first“; Apple uvodi SwiftUI

2021
Jetpack Compose 1.0 — deklarativni UI

2023 →
AI modeli na samom uređaju (npr. Gemini Nano)

Tamna polja = alati koje koristimo u ovom kursu: Kotlin + Jetpack Compose.

Proprietary

ARHITEKTURA

3-slojna arhitektura mobilnog računarstva

SLOJ 1

Prezentacijski

Aplikacija na uređaju: UI, unos, senzori

Prisustvo: ekran „Prijavi se“, skeniranje QR koda

→

SLOJ 2

Aplikacijski

Poslovna logika i servisi (API, oblak)

Prisustvo: provjera koda, vremena i lokacije; prijava korisnika

→

SLOJ 3

Podatkovni

Pohrana i pristup podacima

Prisustvo: baza prijava u oblaku + lokalni keš za rad bez mreže

Svaki sloj se može mijenjati nezavisno — isti backend može služiti Android, iOS i web klijenta.

Proprietary

MOBILNI EKOSISTEM

Aplikacija živi u ekosistemu, ne sama

Operativni sistemi

Android, iOS — i njihove verzije na hiljadama uređaja

Proizvođači i operateri

Samsung, Google, Apple, Xiaomi… mreže i paketi

Prodavnice aplikacija

Google Play, App Store — distribucija, pravila, pregled aplikacija

Servisi u oblaku

Firebase, push notifikacije, mape, plaćanja, AI API-ji

Alati za razvoj

IDE, SDK, emulatori, biblioteke (Jetpack)

Korisnici i developeri

Recenzije, zajednica, dokumentacija

Proprietary

DVA GLAVNA OPERATIVNA SISTEMA

Android i iOS

	Android	iOS
Uređaji	Hiljade modela, svih veličina	Samo iPhone i iPad
Jezik	Kotlin (ranije Java)	Swift (ranije Objective-C)
UI framework	Jetpack Compose	SwiftUI
IDE	Android Studio (Win, Mac, Linux)	Xcode (samo macOS)
Otvorenost	Open source jezgro (AOSP)	Zatvoren, strogo kontrolisan
Distribucija	Google Play + drugi izvori	App Store

Proprietary

PRISTUPI RAZVOJU

Četiri pristupa razvoju

Pristup	Tehnologije	Prednosti	Nedostaci
Nativni	Kotlin + Compose; Swift + SwiftUI	Najbolje performanse, puni pristup hardveru	Dva odvojena koda za dvije platforme
Cross-platform	Flutter, React Native, Kotlin Multiplatform, .NET MAUI	Jedan kod (ili logika) za više platformi	Sloj apstrakcije, sporiji pristup novim API-jima
Web / PWA	HTML, CSS, JavaScript	Bez instalacije, radi u browseru	Ograničen pristup senzorima i pozadinskom radu
Low-code	Builderi aplikacija, AI generatori	Brz prototip bez programiranja	Malo kontrole, zavisnost od platforme

Proprietary

CROSS-PLATFORM DANAS

Ko je ostao, a ko je otišao

Flutter · Dart

Google; vlastiti rendering UI-a, Hot Reload

React Native · JS/TS

Meta; nativne komponente, Fast Refresh

Kotlin Multiplatform · Kotlin

JetBrains; dijeli logiku, UI po izboru (Compose Multiplatform)

.NET MAUI · C#

Microsoft; nasljednik Xamarina

⚠ **Ugašeno:** Adobe PhoneGap (2020) · Xamarin (podrška do maja 2024) · Sencha Touch

Proprietary

RAZVOJNA OKRUŽENJA

Šta nam treba za razvoj

IDE i SDK

- **Android Studio** + Android SDK
- Xcode (iOS, samo macOS)
- VS Code (Flutter, React Native)
- AI asistenti u IDE-u (npr. Gemini u Android Studiju)

Gdje pokrećemo

- Emulator (Android Virtual Device)
- Fizički uređaj preko USB-a ili Wi-Fi-ja (ADB)
- Različite veličine ekrana i verzije OS-a

Zadatak za lab

- Instalirati Android Studio
- Napraviti emulator
- Uključiti Developer options na svom telefonu

Proprietary

FAKTORI U RAZVOJU MOBILNIH APLIKACIJA

Mobilno nije „mali desktop“

Mobilno softversko inženjerstvo

- Fragmentacija: mnogo uređaja i verzija OS-a
- Životni ciklus aplikacije koju OS može prekinuti
- Ažuriranja idu kroz prodavnicu i pregled

Ograničenja uređaja

- Baterija, memorija, procesor
- Nepouzdana mreža — rad offline
- Dozvole i privatnost (kamera, lokacija)

Korisnički interfejs

- Mali ekran, unos dodirom
- Različite veličine, rotacija, sklopivi ekrani
- Pristupačnost i lokalizacija (bs/en)

NOVE DIMENZIJE MOBILNOG RAČUNARSTVA

Šta se mijenja upravo sada

AI na uređaju

Lokalni modeli: sažeci, prepoznavanje, podaci ostaju na telefonu

AI u razvoju

AI piše kod — developer ga mora znati čitati, provjeriti i popraviti

Novi formati

Sklopivi telefoni, tableti, satovi, auto, TV — prilagodljiv UI

Senzori i IoT

Telefon kao centar: zdravlje, pametna kuća, BLE uređaji

Privatnost

Strože dozvole, pravila prodavnica, GDPR

5G i offline-first

Brže mreže, ali aplikacija mora raditi i bez njih

Proprietary

NAŠ IZBOR

Zašto u ovom kursu nativni Android?

01 **Tržište:** oko tri četvrtine telefona u BiH koristi Android

02 **Hardver:** kamera, lokacija, senzori i notifikacije bez posrednika

03 **Moderni alati:** Kotlin + Jetpack Compose — službeni put za Android

04 **Prenosivo znanje:** Kotlin vodi i prema Kotlin Multiplatformu i backendu

05 **Bez Maca:** Android Studio radi na Windowsu, Linuxu i macOS-u

Proprietary

KROZ CIJELI SEMESTAR

Zajedno gradimo aplikaciju za evidenciju prisustva

Compose UI | QR prijava | Provjera lokacije | Više prijava po predavanju | Firebase | Rad offline

Spiskovi odlaze u historiju — a vi učite na stvarnom problemu.

Proprietary

PAUZA · ZATIM DIO 2

Uvod u Kotlin

1. dio — osnove jezika

Otvorite play.kotlinlang.org i pratite primjere uživo

Proprietary

KOTLIN

Zašto Kotlin?

Koncizan
Manje koda za istu stvar nego u Javi

Siguran
Null sigurnost u sistemu tipova

Kompatibilan
Radi zajedno s Java kodom i bibliotekama

Android-first
Preporučeni jezik za Android od 2019.

Multiplatform
Android, iOS, backend, web

Alati
JetBrains pravi i jezik i IDE osnovu

Zanimljivost: ime je dobio po ostrvu Kotlin u Finskom zaljevu, kod Sankt Peterburga. A jezik je, kažu, bez laktoze, šećera i glutena.

Proprietary

OSNOVE

Hello, world!

- **main** je ulazna tačka — funkcija na najvišem nivou, bez klase
- **package** organizuje kod u logičke cjeline
- **print** ispisuje, **println** ispisuje i prelazi u novi red
- Nema tačke-zareza na kraju linije

Vježbajte: play.kotlinlang.org ili Scratch file u Android Studiju

```
// Ulazna tačka programa
fun main() {
    println("Zdravo, RMAS!")
}

// Varijanta s argumentima
fun main(args: Array<String>) {
    println("Argumenata: ${args.size}")
}
```

Proprietary

VARIJABLE

val, var i const val

- **val** — nepromjenljiva referenca
- **var** — promjenljiva
- **const val** — konstanta poznata pri kompajliranju, VELIKIM SLOVIMA
- Statički tipovi: kompajler zaključi tip, a on se više ne mijenja

Pravilo: val po defaultu, var samo kad mora.

```
const val MAX_STUDENATA = 60

fun main() {
  val ime = "Sanja"   // String
  var bodovi = 10    // Int
  bodovi = 12        // OK
  // ime = "Amra"    // Greška:
  // val cannot be reassigned

  var sirina: Int = 12
  val duzina: Double = 2.5
}
```

Proprietary

TIPOVI PODATAKA

Osnovni tipovi i dvije zamke

Tip	Primjer
Int / Long	42 / 42L
Double / Float	3.14 / 3.14f
Boolean	true, false
Char	'A'
String	"Kotlin"

```
// Zamka 1: cjelobrojno dijeljenje
1 / 2       // 0
1.0 / 2     // 0.5

// Zamka 2: nema implicitne konverzije
val i: Int = 6
// val b: Byte = i  // type mismatch
val b: Byte = i.toByte()

val milion = 1_000_000 // čitljivo
```

Proprietary

STRINGOVI

String template-i

- **$ime** — ubacuje vrijednost varijable
- **${izraz}** — ubacuje rezultat izraza
- **"""..."""** — tekst u više linija, bez escape sekvenci

Pitanje: šta ispisuje druga println linija i zašto?

```
val psi = 3
val macke = 2
println("Imam $psi psa i $macke mačke")
// Imam 3 psa i 2 mačke

val s = "abc"
println("$s.length je ${s.length}")
//

val tekst = """
    Više linija
""".trimIndent()
```

Proprietary

KONTROLA TOKA

if je izraz

- **if** vraća vrijednost — možemo je dodijeliti varijabli
- Zato Kotlin nema ternarni operator **? :**
- Opseg **1..100** i operator **in** za provjeru pripadnosti

```
val gosti = 30
val opis = if (gosti == 0) {
    "Nema gostiju"
} else if (gosti < 20) {
    "Mala grupa"
} else {
    "Velika grupa"
}
println(opis)  // Velika grupa

if (gosti in 1..100) println(gosti)
```

Proprietary

KONTROLA TOKA

when umjesto switch

- Grane mogu biti vrijednosti, opsezi ili uslovi
- Kao i **if**, i **when** može vratiti vrijednost
- Kad je izraz, **else** je obavezan
- Kasnije: when nad sealed klasama u Compose UI-ju

```
val rezultati = 50
when (rezultati) {
    0 -> println("Nema rezultata")
    in 1..39 -> println("Ima rezultata")
    else -> println("Puno rezultata!")
}

val cijena = 450
val popust = when (cijena) {
    in 0 until 300 -> 0
    in 300 until 1500 -> 5
    else -> 15
}  // popust = 5
```

Proprietary

KONTROLA TOKA

Petlje i opsezi

- **for … in** — bez ručnog brojača
- **..** uključuje kraj, **until** ne
- **downTo**, **step**
- **withIndex()** daje indeks i element
- **while**, **do-while**, **repeat**

```
val ljubimci = listOf("pas", "mačka", "zec")
for (l in ljubimci) print("$l ")
for ((i, l) in ljubimci.withIndex()) {
    println("$i: $l")
}

for (i in 1..5) print(i)       // 12345
for (i in 0 until 3) print(i)  // 012
for (i in 5 downTo 1) print(i) // 54321
for (i in 3..6 step 2) print(i)// 35

var n = 0
while (n < 3) n++
repeat(2) { print("Hej! ") }
```

Proprietary

KOLEKCIJE

Liste i nizovi

- **listOf** — samo za čitanje
- **mutableListOf** — dodavanje i brisanje
- **arrayOf** — fiksna veličina, elementi promjenljivi
- **val** zaključava referencu, ne sadržaj

U Android kodu najčešće koristimo List.

```
val sale = listOf("A1", "A2", "B3")
// sale.add("C1")  // Greška: read-only

val prisutni = mutableListOf("Amar")
prisutni.add("Lejla")
prisutni.remove("Amar")  // true

val niz = arrayOf("pas", "mačka")
niz[0] = "zec"
println(niz.contentToString())
// [zec, mačka]

val svi = intArrayOf(1, 2) + intArrayOf(3)
```

Proprietary

FUNKCIJE

fun, default i imenovani argumenti

- Ključna riječ **fun**; parametri imaju tip i nepromjenljivi su
- Jednoizrazna funkcija: **= izraz**
- **Default** vrijednost čini parametar opcionalnim
- **Imenovani** argumenti: jasniji poziv, bilo koji redoslijed

```
fun pozdrav(ime: String): String {
    return "Zdravo, $ime!"
}
fun kvadrat(x: Int) = x * x

// default i imenovani argumenti
fun prijava(
    student: String,
    sedmica: Int = 1,
    kasni: Boolean = false
) = "$student, sedmica $sedmica, kasni: $kasni"

prijava("Amar")
prijava("Lejla", kasni = true)
```

Proprietary

VJEŽBA · 5 MINUTA

Četiri Java funkcije → jedna u Kotlinu

```
// Java
String foo(String name, int number, boolean toUpperCase) {
    return (toUpperCase ? name.toUpperCase() : name) + number;
}
String foo(String name, int number) { return foo(name, number, false); }
String foo(String name, boolean toUpperCase) { return foo(name, 42, toUpperCase); }
String foo(String name) { return foo(name, 42); }
```

Napišite **jednu** Kotlin funkciju tako da se svi pozivi desno kompajliraju.

```
foo("a")                    // a42
foo("b", number = 1)        // b1
foo("c", toUpperCase = true)// C42
```

Proprietary

NULL SIGURNOST

String i String? su različiti tipovi

- NullPointerException je najčešći pad aplikacija u Javi
- U Kotlinu tip po defaultu **ne može** biti null
- **?** iza tipa dozvoljava null
- Kompajler traži provjeru prije upotrebe — nakon nje radi **smart cast**

```
val ime: String = "Lejla"
// val x: String = null  // Greška

val nadimak: String? = null
// nadimak.length        // Greška

if (nadimak != null) {
    println(nadimak.length)
    // ovdje je nadimak String
} else {
    println("Nema nadimka")
}
```

Proprietary

NULL SIGURNOST

?. ?: i let

- **?.** safe call — vraća null umjesto da pukne; radi i u lancu
- **?:** Elvis — vrijednost ako je lijevo null; desna strana se računa samo tada
- **?.let { }** — izvrši blok samo za ne-null vrijednost

```
// safe call u lancu
println(student.grupa?.asistent?.ime)

// Elvis: zamjenska vrijednost
val prikaz = nadimak ?: "Bez nadimka"

// Elvis s return i throw
fun email(id: String): String? {
    val s = nadjiStudenta(id) ?: return null
    return s.email ?: throw Exception("…")
}

nadimak?.let { println(it) }
```

Proprietary

NULL SIGURNOST

!! i TODO(): kompajlira se, ali puca

- **!!** pretvara u ne-null tip i baca NPE ako je vrijednost null
- **TODO()** uvijek baca NotImplementedError — korisno za skicu koda

⚠ **!!** u AI-generisanom kodu = crvena zastavica. Pitajte: zašto ovo ne može biti null?

```
// „vjerujem da nije null“
val d = nadimak!!.length
// NullPointerException ako je null

// kompajlira se, puca pri pozivu
fun nadjiStudenta(id: String): Student? =
    TODO("Pronađi studenta $id")
```

Proprietary

LAMBDA IZRAZI

Funkcija kao vrijednost

- Lambda: **{ parametri -> tijelo }**
- Zadnja lambda ide **van zagrada** (trailing lambda)
- Jedan parametar: implicitno **it**

Ovo je osnova Compose-a:
Button(onClick = { ... }) { Text("Prijava") }

```
val zbir: (Int, Int) -> Int = { x, y -> x + y }
val puta = { x: Int, y: Int -> x * y }
println(zbir(2, 3))    // 5

val brojevi = listOf(1, 2, 3, 4)
// trailing lambda
brojevi.fold(1) { acc, e -> acc * e } // 24

// jedini argument: bez zagrada
run { println("Very cool") }

brojevi.filter { it % 2 == 0 } // [2, 4]
```

Proprietary

VJEŽBA ČITANJA KODA · U PAROVIMA, 7 MINUTA

AI je napisao ovaj kod. Šta ne valja?

- Pronađite bar **3 problema**
- Za svaki: kada pukne ili vrati pogrešan rezultat?
- Kako biste ga popravili?

Kod se kompajlira bez greške — a to nije isto što i ispravan kod.

```
fun prosjek(ocjene: List<Int>): Int {
    var suma = 0
    for (i in 0..ocjene.size) {
        suma += ocjene[i]
    }
    return suma / ocjene.size
}

fun ispisi(ime: String?) {
    println("Student: " + ime!!.uppercase())
}
```

Proprietary

RJEŠENJE

Četiri problema, jedan popravak

Problem	Posljedica
0..ocjene.size	Opseg uključuje size → IndexOutOfBoundsException
suma / ocjene.size	Cjelobrojno dijeljenje: prosjek 4.5 postaje 4
Prazna lista	Kad se petlja popravi: dijeljenje nulom → ArithmeticException
ime!!	NullPointerException kad ime nije poznato

```
fun prosjek(ocjene: List<Int>): Double =
    if (ocjene.isEmpty()) 0.0 else ocjene.sum().toDouble() / ocjene.size

fun ispisi(ime: String?) = println("Student: ${ime?.uppercase() ?: "nepoznat"}")
```

Proprietary

Šta slijedi

LABORATORIJSKE VJEŽBE

Android Studio, emulator, prvi Compose projekat i Kotlin vježbe

SLJEDEĆE PREDAVANJE

Kotlin II: klase, data i sealed klase, kolekcije, ekstenzije — zatim osnove Compose-a

PROJEKAT

Razmislite o ideji za svoju aplikaciju

LITERATURA

kotlinlang.org/docs · play.kotlinlang.org/byExample · Kotlin Koans · developer.android.com/courses

Hvala!